

Non-Leaking Concurrent Objects

Hagit Attiya ✉ 

Technion, Haifa, Israel

Rotem Oshman ✉ 

Tel-Aviv University, Tel-Aviv, Israel

Noa Schiller ✉ 

Tel-Aviv University, Tel-Aviv, Israel

Corentin Travers ✉ 

Aix Marseille Univ, CNRS, LIS, Marseille, France

Abstract

Abstract specifications of concurrent objects determine which values operations may return, but they also implicitly constrain which information operations may know, for example the arguments of other operations that do not affect their outcome, or even whether such operations occurred. Concrete implementations, while correct with respect to the abstract specification, may nonetheless expose additional information through their internal coordination mechanisms.

We introduce a framework for reasoning about information leakage in concurrent implementations. The framework uses epistemic logic to compare what a process may know under an abstract specification with what it may know in a concrete implementation, using the abstract object itself as the reference for permissible observations. This yields several notions of *non-leaking* implementations.

Using this framework, we investigate both the possibilities and limitations of non-leaking implementations. We present fully-non-leaking wait-free implementations of multi-valued registers and bounded max registers, but show that a fully-non-leaking unbounded max register cannot be implemented in a wait-free manner from finite-state base objects. We then consider a weaker guarantee, obtaining argument-non-leaking implementations of stacks, queues, and approximate agreement. These results demonstrate that non-leakage guarantees are often compatible with correctness and progress requirements, while also indicating their limitations.

2012 ACM Subject Classification Theory of computation → Distributed algorithms

Keywords and phrases non-leaking implementations, epistemic logic, knowledge, concurrent objects, decision tasks

Funding *Hagit Attiya*: Supported by the Israel Science Foundation (22/1425 and 25/1849).

Rotem Oshman: Supported by NSF-BSF 2022699.

Noa Schiller: Supported by NSF-BSF 2022699.

Corentin Travers: Supported by ANR-23-PECL-0009 TRUSTINCloudS.

Contents

1	Introduction	2
2	Definitions	4
2.1	Labeled Transition Systems and Refinement	4
2.2	Non-Leaking in Terms of Epistemic Logic	5
2.3	Non-Leaking of Operations of the Abstract Object	6
3	A Wait-Free Fully-Non-Leaking N-Valued Single-Writer Register	8
4	A Wait-Free Fully-Non-Leaking Bounded Max Register	10
5	No Wait-Free Unbounded Fully-Non-Leaking Max Register	13

6	Argument-Non-Leaking Wait-Free Stack and Lock-Free Queue	15
6.1	Wait-Free Stack	15
6.2	Lock-free Queue	18
7	Input-Non-Leaking Wait-Free Approximate Agreement	19
7.1	Defining Input-Non-Leaking Approximate Agreement	19
7.2	Schenk's Algorithm (Slightly Modified)	19
7.3	Proof of Correctness and Non-Leakage	21
8	Conclusions	22
A	Wait-Free Unbounded Max Register from Responseless CAS	24

1 Introduction

Abstract specifications of concurrent objects describe the observable behavior of operations: which values operations may return and how these values relate to one another. At the same time, such specifications implicitly constrain what an operation *does not observe*. For example, reading a variable in the abstract object does not reveal which values were written and later overwritten; popping an element from a stack does not reveal which other elements were concurrently pushed. In this sense, abstract objects specify not only the information that must be preserved by an implementation, but also information that is intentionally left hidden. Concrete implementations, even when correct with respect to the abstract behavior, may nonetheless expose this additional information through their internal coordination or contention mechanisms.

To see how the abstract specification determines which information an operation is allowed to infer, consider two examples. In a *counter* object supporting FETCH&INC, if one operation returns 5 and a later operation returns 8, then in every execution of the abstract counter there are exactly two intervening FETCH&INC operations returning 6 and 7. Thus, it is legitimate for a concrete implementation to reveal the existence of these intervening operations, since it is already implied by the abstract behavior. In contrast, in a *max register*, if one READMAX returns 5 and a later READMAX returns 8, the abstract specification does not imply that any intermediate writemax operations occurred; the increase may be due to a single writemax(8). Revealing the existence or arguments of such intermediate writemax would therefore expose information that is not mandated by the abstract object. For example, if a leaky max register is used for a *highest-bidder wins* auction, it can inadvertently reveal the price offers of non-winning bids, which does not happen with the abstract max register. These examples illustrate how the abstract specification itself determines which information may or may not be observable in a correct implementation.

This paper initiates the study of concurrent implementations that preserve these information-hiding guarantees. To formalize such guarantees, we introduce a semantic framework based on *epistemic logic*, a standard formalism for reasoning about knowledge [12]. At a high level, the framework compares what a process may learn under an abstract specification with what it may learn in a concrete implementation. The key idea is that the abstract object itself serves as the reference for permissible information exposure and the implementation should not reveal more than that. That is, a non-leaking implementation does not reveal information about other operations beyond what is logically implied by the abstract specification and by the operation's own return value. Returning to the examples

above, the existence of intermediate FETCH&INC operations is implied by the abstract counter semantics, whereas the existence of intermediate WRITEMAX operations is not.

In different situations we might want—or be able—to hide different types of information. For example, a process may learn that an operation occurred, but not which arguments it carried. To capture that, the framework is parameterized by the type of knowledge under consideration. In particular, we define two notions of *non-leaking* implementations. In a *fully-non-leaking* implementation, processes learn nothing about individual operations beyond what is implied by the abstract specification. In particular, they do not learn whether other operations occurred, nor do they learn their arguments or return values. A weaker notion, *argument-non-leaking*, permits processes to learn that operations have occurred while preventing them from learning the identity of the invoking process or the arguments and return values of the operation. More generally, the framework allows one to express and reason about a spectrum of information-hiding guarantees for concurrent objects.

One of the central purposes of abstraction is to hide implementation details and allow clients to reason about an object solely through its specification. When an implementation reveals information that is not determined by the abstract object, it breaks this abstraction boundary: a client may distinguish between behaviors that are equivalent at the abstract level, and may know information through implementation mechanisms intended to deal with concurrency and contention. A non-leaking implementation preserves the information-hiding guarantees implicit in the abstract specification by ensuring that operations do not acquire knowledge beyond what the abstraction itself exposes.

We investigate possibilities and limitations of information-hiding implementations. We present fully-non-leaking wait-free implementations of *multi-valued registers* and *bounded max registers*, but show that an *unbounded* fully-non-leaking max register cannot be implemented in a wait-free manner from finite-state base objects. We then consider the weaker guarantee, and present an argument-non-leaking wait-free stack and lock-free queue. We also extend the framework to decision tasks, obtaining an argument-non-leaking wait-free approximate agreement algorithm. These results show that information-hiding guarantees can often be achieved without sacrificing standard correctness and progress properties, while also identifying inherent limitations of the strongest form of information hiding.

Related Work. Traditional correctness notions for concurrent implementations are based on *refinement* [17], and they characterize *admissible executions and return values*. They do not constrain the information that processes can observe: for example, a *linearizable* implementation [15] may expose internal state or intermediate results not determined by the abstract specification. Leaking this information to the *scheduling adversary* leads to the surprising behaviors observed in [14] (which are addressed by imposing the stronger requirement of *strong linearizability*, preventing the adversary from being able to “change the linearization” after learning internal information). Our work goes in a different direction: our goal is to protect intermediate states from observation by *other processes*, so that they do not learn specific information not revealed by the abstract specification.

Our epistemic framework for information gained by processes is based on *indistinguishability* of executions from the perspective of a single process, a standard technique in distributed computing [7]. Recent work on *auditing under curiosity* [3] also employs indistinguishability-based reasoning to limit information disclosure, but is restricted to audit and logging mechanisms. Our approach instead uses the abstract object specification as the reference for allowable information exposure for *arbitrary* objects.

Another closely related work studies *concurrent history independence* [4, 5], where an

external observer may inspect the entire memory state at quiescent points and should not infer information beyond what is implied by the current state. Under this notion, even a single-writer N -valued register cannot be implemented from base objects with fewer than N states [4]. In contrast, we present a fully-non-leaking implementation of such a register from binary registers. This separates non-leaking and history-independent implementations: the former reason about what an executing operation can infer during execution, while the latter considers what an external observer can infer from quiescent memory states.

In *amnesic* register implementations [10], old values are eventually deleted from memory. This definition takes a perspective similar to history independence, as it considers the state of the entire memory configuration rather than the execution of a particular operation.

There is extensive work on information hiding in other settings. *History independence* studies what information about past operations can be inferred from the memory representation of a data structure [20], while *information-flow security* studies how secret inputs may influence observable outputs [13]. Related notions of observational or contextual equivalence and refinement appear in programming languages, where they are used to reason about substitutability [19]. These approaches typically consider external observers, final memory states, or explicit security policies. In contrast, we study what a process can infer from its own observations while executing operations, using the abstract object specification as the reference for permissible information exposure.

We also note a connection to *differential privacy* [11]. Our setting differs in that the observer is an operation executing the implementation (rather than an external analyst), and the relevant changes are guided by the abstract object semantics. This perspective suggests quantitative variants of non-leaking properties, obtained by requiring the distribution of an operation's view to change only slightly under such semantic neighbors. Non-leaking decision tasks are reminiscent of simulation-based definitions in secure multi-party computation [23], though our setting is non-cryptographic since we assume a weaker adversary.

2 Definitions

2.1 Labeled Transition Systems and Refinement

We use labeled transition systems [16] to model both abstract and concrete objects (specifications and implementations, respectively). A *labeled transition system* (LTS) (Q, Σ, s_0, δ) over the possibly-infinite alphabet Σ is a possibly-infinite set Q of states with initial state $s_0 \in Q$, and a transition relation $\delta \subseteq Q \times \Sigma \times Q$. The i th symbol of a sequence $\tau \in \Sigma^*$ is denoted τ_i , and ϵ is the empty sequence. An LTS is *deterministic* if for any state s and any sequence $\tau \in \Sigma^*$, there is at most one state s' such that $s \xrightarrow{\tau} s'$.

A finite *execution* of a LTS is an alternating sequence of states and transition labels (also called *actions*) $\rho = s_0, a_0, s_1 \dots a_{k-1}, s_k$ for some $k > 0$ such that $(s_i, a_i, s_{i+1}) \in \delta$ for each $0 \leq i < k$. The projection $\tau|_\Gamma$ of a sequence τ is the maximum subsequence of τ over alphabet $\Gamma \subseteq \Sigma$; this extends to sets of sequences as usual. A *trace* is the projection $\rho|_\Sigma$ of an execution ρ . $E(A)$ is the set of executions of an LTS A , while $T(A)$ is the set of its traces.

Throughout the paper, we assume a fixed set of processes P . An *object* is a *deterministic* LTS over alphabet $Call \cup Ret \cup \Sigma_o$ where $Call$, resp., Ret , is the set of call, resp., return, actions, and Σ_o is an alphabet of internal actions. Formally, a call action $call(m, d, p, k)$, resp., a return action $ret(m, d, p, k)$, combines a method m and argument, resp., return value, d with a process identifier p and an operation identifier k . Operation identifiers are used to pair call and return actions. We assume that the traces of an object satisfy standard well-formedness properties, e.g., return actions correspond to previous call actions.

A *configuration* of the system contains the states of all shared base objects and processes. In an *initial* configuration, base objects and processes are in their initial states.

Given a standard description of an object implementation as a set of methods, its LTS represents the executions of its most general client (that may call methods in any order and from any thread). It is convenient to conflate the *states* of the LTS with its *executions* by assuming w.l.o.g. that the state space is simply the set of executions. Because we consider only deterministic implementations, each execution ends in a unique configuration, so each state of the LTS corresponds to a unique configuration of the system. The transitions of the LTS correspond to statements in the method bodies (in which case they are labeled by internal actions in Σ_o), or call and return actions. A trace τ of an object O projected over call and return actions is called a *history* of O , and it is denoted by $hist(\tau)$. The set of histories admitted by an object O is denoted by $H(O)$.

Call and return actions $call(m, _, p, k)$ and $ret(m, _, p, k)$ are called *matching* when they contain the same process and operation identifiers p, k . A call action is *unmatched* in a history h when h does not contain the matching return. We often refer to a call action $call(m, v_1, p, k)$ and its matching return action $ret(m, v_2, p, k)$, if it exists, as an *operation* with argument v_1 and return value v_2 , denoted $op(m, p, v_1, v_2)$. A history h is *sequential* if every call $call(m, _, p, k)$ is immediately followed by the matching return $ret(m, _, p, k)$.

The *view* of a process p in an execution γ is the projection of γ onto actions taken by process p (i.e., operation calls and returns where the process identifier is p , and internal actions taken by process p).

An object C *refines* object A if all histories of C are histories of A , i.e., $H(C) \subseteq H(A)$. In this case we often refer to C as the *concrete object*, and to A as the *abstract object* that C refines. A *refinement mapping* from C to A is a mapping $r : H(C) \rightarrow H(A)$; clearly, a refinement mapping from C to A exists iff C refines A .

In this paper, the concrete objects are implementations of abstract objects in the standard asynchronous shared-memory model [8] with n processes, $P = \{p_0, \dots, p_{n-1}\}$. Processes communicate with each other by applying *primitive* operations to shared *base objects*, and these are the actions of the concrete object. A concrete *implementation* C of an abstract object A is an algorithm that provides a local code for each process p , which specifies the primitive operations on base objects p applies and local computations p performs in order to return a response when it invokes an operation of A .

Linearizability [15] is essentially refinement when the abstract object A is sequential. Intuitively, C is a *linearizable* implementation of A if each operation appears to take effect *atomically* at some time between its invocation and response, hence operations' real-time order is maintained. We consider the standard *wait-free* and *lock-free* progress conditions.

Indistinguishable executions and “no forgetting”. Two executions γ and γ' are *indistinguishable* to process p , denoted $\gamma \sim_p \gamma'$, if it has the same view in both executions. This will be important in the context of our definition of non-leaking implementations: it means that processes can use past observations to try to extract information about other processes.

2.2 Non-Leaking in Terms of Epistemic Logic

Let $L = (\Sigma, S, \delta)$ be an LTS representing a concurrent object for a set of processes P . In addition, let AP be a set of atomic propositions, and let $v : S \times AP \rightarrow \{true, false\}$ be an evaluation function indicating for each state $s \in S$ and atomic proposition p whether p holds true in state s . We denote $s \models p$ if $v(s, p) = true$.

Syntactically, the formulas of *epistemic logic* are inductively defined by the base set AP and the operators $\{f_\wedge(\cdot, \cdot), f_\vee(\cdot, \cdot), f_\neg(\cdot)\} \cup \{f_{K_p}(\cdot) : p \in P\}$, where $f_\circ(\alpha, \beta) = (\alpha \circ \beta)$ for $\circ \in \{\wedge, \vee\}$ and $f_\circ(\alpha) = \circ(\alpha)$ for $\circ \in \{\neg\} \cup \{K_p : p \in P\}$. We omit the parentheses when the order of operations is clear, with $\neg$ and K_p taking precedence over $\wedge, \vee$.

The semantics of epistemic logic is also defined inductively, with the base case (atomic formulas) already defined above, and the Boolean operators $\wedge, \vee, \neg$ evaluated as usual. The knowledge operator K_p is evaluated as follows: $s \models K_p \varphi$ iff for every state $s' \in S$ such that $s' \sim_p s$, we have $s' \models \varphi$. (Note that this includes state s itself. Here, $\sim_p$ is the indistinguishability relation defined above.)

Recall that we conflate the *states* of an LTS representing a concurrent object with the *executions* of the object, so we will frequently write expressions such as $\beta \models \varphi$ to indicate that formula φ holds in the state of the LTS that corresponds to execution β .

Let $L_{abs} = (\Sigma_{abs}, S_{abs}, \delta_{abs})$, $L_{conc} = (\Sigma_{conc}, S_{conc}, \delta_{conc})$ be two LTSs enriched with the same set of atomic propositions AP. Fix a set of atomic propositions $A \subseteq AP$.

If L_{conc} refines L_{abs} (as defined above), then we say that it is a *A-non-leaking implementation of L_{abs}* if for every $a \in A$, process p , and executions α and β of L_{abs} and L_{conc} (resp.) such that $r(\beta) = \alpha$ and process p has no pending operations in β ,

If $\alpha \not\models K_p a$ then $\beta \not\models K_p a$.

► **Proposition 1.** *Let $A_1 \subseteq A_2$ be sets of atomic propositions. If an implementation is A_2 -non-leaking, then it is also A_1 -non-leaking.*

The framework that we introduced can capture non-leaking of *any* class of facts (i.e., atomic propositions), but in this paper we focus on leakage of individual operations performed by other processes, and/or the arguments and return values of those operations, defined next.

2.3 Non-Leaking of Operations of the Abstract Object

Let Met be the set of methods supported by the object, and for each $m \in Met$, let Arg_m, Ret_m be the possible arguments and return values of method m (resp.). We slightly abuse notation by letting $op(m, p, v_1, v_2)$ be an atomic proposition which holds true in state γ of the LTS if and only if $op(m, p, v_1, v_2)$ appears (and is complete) in the execution γ (for $m \in Met, p \in P, v_1 \in Arg_m, v_2 \in Ret_m$).

We define two sets of atomic propositions, each reflecting a different type of knowledge about operations that have occurred in the execution:

- Knowledge that a specific operation was invoked:

$$A_{op} = \{ \exists p \in P \exists a \in Arg_m \exists r \in Ret_m . op(m, p, a, r) \quad : \quad m \in Met \}.$$

- Knowledge about the invoking process, arguments and/or return value:

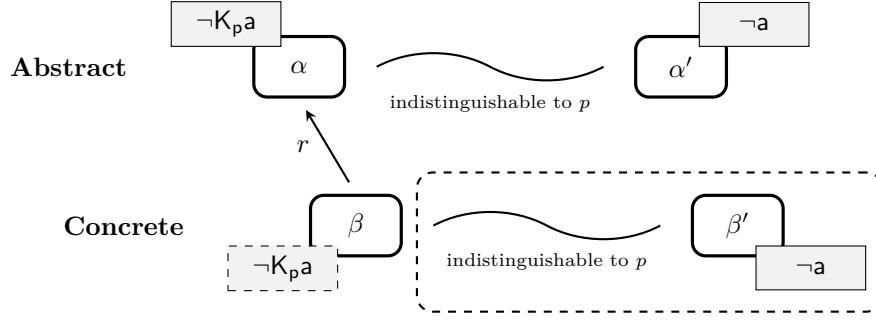
$$A_{arg} = \{ \exists (p, a, r) \in V . op(m, p, a, r) \quad : \quad m \in Met, V \subsetneq P \times Arg_m \times Ret_m \}.$$

An implementation is called *fully-non-leaking* if it is $(A_{op} \cup A_{arg})$ -non-leaking, and is *argument-non-leaking* if it is A_{arg} -non-leaking.

Given a set $V \subseteq P \times Arg_m \times Ret_m$, we use the following short-hand notation. Let

$$op(m, V) = \exists (p, a, r) \in V . op(m, p, a, r)$$

be an atomic proposition asserting that an operation $op(m, p, a, r)$ with $(p, a, r) \in V$ has occurred. Using this notation we have $A_{op} = \{op(m, P \times Arg_m \times Ret_m) : m \in Met\}$, and $A_{arg} = \{op(m, V) : m \in Met, V \subsetneq P \times Arg_m \times Ret_m\}$. If an operation has no arguments and/or no return value, we may omit them from our notation.



■ **Figure 1** Proving that an implementation is non-leaking with respect to atomic proposition a . Dashed boxes indicate claims that must be established: namely, to prove that $\beta \not\models K_p a$, we must exhibit a p -indistinguishable execution β' such that $\beta' \not\models a$.

A useful proof technique. Recall that the assertion $\gamma \not\models K_p a$ means that there exists a p -indistinguishable execution γ' such that $\gamma' \not\models a$. Thus, proving that an implementation is A -non-leaking boils down to the following: given an atomic proposition $a \in A$, abstract executions α, α' such that $\alpha \sim_p \alpha'$ and $\alpha' \not\models a$, and a concrete execution β that refines α , we must exhibit a concrete execution β' such that $\beta \sim_p \beta'$ and $\beta' \not\models a$ (see Fig. 1).

To find β' , it is helpful to consider a “simplest possible” abstract execution $\alpha'' \sim_p \alpha'$ such that $\alpha'' \not\models a$, and try to find a concrete execution β' that “matches it”. In our case, since the atomic propositions that we consider deal with the presence of operations in the trace, “simplest” translates to having as few operations as possible. Thus, for an abstract execution γ , let us define $\min_p(\gamma)$ to be a “minimal p -indistinguishable version of γ ”: $\min_p(\gamma)$ contains a subset of the operations in γ , such that $\min_p(\gamma) \sim_p \gamma$, but we cannot remove any operation from $\min_p(\gamma)$ without violating p -indistinguishability. (There may be more than one such execution, in which case we fix one arbitrarily.) Working with this minimal execution allows us to prove non-leakage without considering each atomic proposition individually, by proving the following relationship between abstract and concrete executions:

► **Lemma 2.** *Let L_{conc} be an implementation of L_{abs} with a refinement mapping r , and assume that for every process p , every pair of abstract executions α, α' (of L_{abs}) such that $\alpha' \sim_p \alpha$, and every concrete execution β (of L_{conc}) such that $r(\beta) = \alpha$, there exists a concrete execution β' (of L_{conc}) such that $\beta' \sim_p \beta$ and β' contains exactly the operations in $\min_p(\alpha')$. Then L_{conc} is a fully-non-leaking implementation of L_{abs} .*

Proof. Let a be a proposition in $A_{op} \cup A_{arg}$ such that $\alpha \not\models K_p a$. We show that $\beta \not\models K_p a$.

Since $\alpha \not\models K_p a$, there is an abstract execution $\alpha' \sim_p \alpha$ such that $\alpha' \not\models a$. Since the atomic proposition a concerns the existence of a specific operation, $\gamma \not\models a$ for any (concrete or abstract) execution γ containing only a subset of the operations in α' . Hence, it also holds that $\min_p(\alpha') \not\models a$. By assumption, there is an execution $\beta' \sim_p \beta$ containing exactly the operations in $\min_p(\alpha')$. This implies that $\beta' \not\models a$, and therefore, $\beta \not\models K_p a$. ◀

Discussion. The definition of A_{arg} is more subtle than A_{op} , and is intended to capture *any* knowledge gained about the invoking process and the arguments and return value of the operation, including relationships between them. For example, it captures the following facts:

- Operation m was invoked by process p with a specific argument a and return value r : we take $V = \{p\} \times \{a\} \times \{r\}$.
- For an operation m that takes one argument and returns a value from the same domain ($Arg_m = Ret_m$), the fact that operation m was invoked by an unknown process, and its

■ **Algorithm 1** Fully-non-leaking wait-free single-writer N -valued register

```

1: Shared objects
2:    $r$ :  $\lceil N/2 \rceil$ -valued register, initialized to 0
3:    $A$ : array  $[0.. \lceil N/2 \rceil - 1]$  of binary registers (each initially 0)
4: function WRITE( $x$ )                                7: function READ( )
5:    $A[\lfloor x/2 \rfloor].\text{write}(x \bmod 2)$               8:    $x \leftarrow r.\text{read}()$ 
6:    $r.\text{write}(\lfloor x/2 \rfloor)$                         9:   return  $2x + A[x].\text{read}()$ 

```

return value was the same as its argument: $V = P \times \{(v, v) : v \in \text{Arg}_m\}$. (Note that we do not learn what the argument or return value *are*, only that they are the same.)

In order to be A_{arg} -non-leaking an implementation must not leak *any* such fact.

We also remark that A_{op} and A_{arg} are incomparable in the sense that an implementation may be A_{op} -non-leaking but not A_{arg} -non-leaking and vice-versa. For example, an implementation of a register may not leak that any particular operation has occurred (and so is A_{op} -non-leaking), but in situations where the abstract specification *allows* learning that a specific operation has occurred, we may learn the arguments of the operation even though the abstract specification does not reveal them (and so the implementation is not A_{arg} -non-leaking). In the other direction, an implementation may leak that certain operations have occurred (and thus not be A_{op} -non-leaking) while not revealing anything about the arguments or return values (and thus be A_{arg} -non-leaking); this is the case for the stack and queue implementations described in Section 6.

3 A Wait-Free Fully-Non-Leaking N -Valued Single-Writer Register

We start with a simple and elegant fully-non-leaking implementation of a single-writer multi-reader N -valued register, from binary registers. Algorithm 1 is a recursive implementation of an N -valued register, from an $\lceil N/2 \rceil$ -valued register r and an array A of $\lceil N/2 \rceil$ binary registers. Recursively implementing r allows using only binary registers.

Each value x of r represents the two values $2x$ and $2x + 1$; the corresponding entry $A[x]$ indicates the parity of the most recent value written among the two. We assume that r and $A[0]$ are initialized to 0, representing that the register value is initially 0. A $\text{WRITE}(x)$ first writes to $A[\lfloor x/2 \rfloor]$ the parity of x , and then announces the new value by writing $\lfloor x/2 \rfloor$ to r . A READ operation accesses the memory in the reverse order of the writes: it first reads a value x from r and a parity p from $A[x]$, returning the final value $2x + p$.

Unfolding the recursive construction leads to a binary tree of binary registers with height $\lceil \log_2 N \rceil$. The writer updates the value bit-by-bit, starting from the least significant bit in the leaves and concluding with the most significant bit at the root.

We linearize the operations as follows. We begin with a $\text{WRITE}(x)$ operation wop . The linearization point for wop is defined as the earliest step in the execution between wop performing $r.\text{write}(\lfloor x/2 \rfloor)$ and any step in which a READ operation reads the value wop writes to $A[\lfloor x/2 \rfloor]$. To linearize READ operations we prove (Lemma 3) that if a READ operation reads the value b from $A[x]$, then either $b = 0$ and $x = 0$, or this value was written by a $\text{WRITE}(2x + b)$ operation. A READ operation rop is linearized after the WRITE operation that writes the value to A which rop reads. If rop reads the initial value from A , it is linearized at the beginning before any WRITE operation.

► **Lemma 3.** *If a READ operation reads the value b from $A[x]$, then either $b = 0$ and $x = 0$, or this value was written by a $\text{WRITE}(2x + b)$ operation.*

Proof. Before a READ operation rop reads $A[x]$, it first reads the value x from r . Conversely, before a WRITE(x) operation writes the value $\lfloor x/2 \rfloor$ to r , it first writes to $A[\lfloor x/2 \rfloor]$.

Consequently, rop either reads the initial values from r and $A[0]$, or it reads a value x that was written by some WRITE($2x$) or WRITE($2x + 1$) operation. As explained above, the ordering of these steps ensures that by the time x is visible in r , the value b read from $A[x]$ is no longer the initial value, but rather the value stored there by a WRITE($2x + b$) operation. ◀

► **Lemma 4.** *Algorithm 1 implements a linearizable single-writer N -valued register.*

Proof. Lemma 3 shows that the linearization satisfies the sequential specification of a register. We show that the linearization preserves the real-time order of non-overlapping operations. The order between WRITE operations is preserved because they are linearized at a step between the invocation and response of the operation.

Consider a READ operation rop . rop cannot precede the WRITE operation wop that it is linearized after, as rop reads a value written by wop . Because all other WRITE operations placed before rop in the linearization precede wop , it follows that rop cannot precede them.

Consider a WRITE operation wop that precedes rop . Then, rop reads from r a value written by wop' where $wop' = wop$ or wop' follows wop (recall all WRITE operations are non-overlapping). Since wop' first writes to A before writing to r , it also follows that rop reads from A a value written by wop'' where $wop'' = wop'$ or wop'' follows wop' . rop is linearized after wop'' , and is therefore also linearized after wop .

Consider two READ operations, rop_1 and rop_2 , such that rop_1 precedes rop_2 in the execution. Let wop be the WRITE operation rop_1 is linearized after.

Note that wop writes to A before rop_1 returns. Since WRITE operations do not overlap, rop_2 reads from r a value written by wop' where $wop' = wop$ or wop' follows wop . Since wop' first writes to A before writing to r , it also follows that rop_2 reads from A a value written by wop'' where $wop'' = wop'$ or wop'' follows wop' . rop_2 is linearized after wop'' , and is therefore also linearized after wop . This implies rop_2 is also linearized after rop_1 . ◀

Next we prove that it is fully-non-leaking. A WRITE operation only writes to memory, so the writer does not learn anything about other operations. The non-leakage property for READ operations follows by observing that a READ operation returns x if and only if it reads $\lfloor x/2 \rfloor$ from r and $x \bmod 2$ from $A[\lfloor x/2 \rfloor]$. These are the only two memory reads a READ operation performs, implying:

► **Lemma 5.** *Algorithm 1 is a fully-non-leaking implementation of a single-writer register.*

Proof. Let α, β be executions of respectively the abstract register and Algorithm 1 such that α is the linearization of β , as defined in the proof of Lemma 4. Let $\alpha' \sim_p \alpha$. We show that there exists a concrete execution $\beta' \sim_p \beta$ that contains exactly the operations in $\min_p(\alpha')$. By Lemma 2, this implies that the algorithm is fully-non-leaking.

We construct β' by iterating over the operations of p in $\min_p(\alpha')$. Since $\min_p(\alpha') \sim_p \alpha$ and α is the linearization of β , both $\min_p(\alpha')$ and β contain the exact same operations by process p , appearing in the exact same order. If the next operation of p is a WRITE operation, we simply append it to β' . We can do this because p only writes during the operation and never reads from memory. Thus, p gains no knowledge about the other operations in β .

If the next operation of p is a READ operation that returns x , then consider the last WRITE operation (if it exists) in β' . If the argument for this WRITE is x , we simply append the READ operation by p to β' . Otherwise, since $\min_p(\alpha')$ respects the sequential specification,

■ **Algorithm 2** Fully-non-leaking wait-free max register with input domain $\{0, \dots, N-1\}$

```

1: Shared objects
2:    $S[0..N-1]$ : array of  $N$  registers, initially 0 except  $S[0] = 1$ 
3:    $I[0..N-1]$ : array of  $N$  registers, initially 0
4: function  $\text{WRITEMAX}(x)$ 
5:    $S[x].\text{write}(1)$ 
6:   for  $y = x-1, x-2, \dots, 0$  do  $I[y].\text{write}(1)$ 
7: function  $\text{READMAX}()$ 
8:   for  $x = N-1, N-2, \dots, 0$  do ▷ Downward scan
9:     if  $I[x].\text{read}() = 1$  then
10:      for  $z = x+1, \dots, N-1$  do ▷ Upward scan
11:        if  $I[z].\text{read}() = 0$  then
12:          for  $y = z-1, z-2, \dots, 0$  do  $I[y].\text{write}(1)$ 
13:          return  $z$ 
14:   if  $S[x].\text{read}() = 1$  then
15:     for  $y = x-1, x-2, \dots, 0$  do  $I[y].\text{write}(1)$ 
16:   return  $x$ 

```

the preceding WRITE operation in $\min_p(\alpha')$ has argument x and is by process $q \neq p$. We first append a complete WRITE(x) by process q , and then append the READ operation by p to β' .

By construction, $\beta' \sim_p \beta$. In addition, due to the minimality of $\min_p(\alpha')$, any operation not executed by p exists solely to justify a value returned by a READ operation by p . Hence, $\min_p(\alpha')$ does not contain any operation that is not included in β' . ◀

4 A Wait-Free Fully-Non-Leaking Bounded Max Register

A max register [2] stores values from a totally ordered domain V and supports two operations, $\text{WRITEMAX}(x)$ and $\text{READMAX}()$, where READMAX returns the largest value previously written by a WRITEMAX operation. In this section, we present a non-leaking implementation of a max register with finite domain $V = \{0, \dots, N-1\}$.

Algorithm 1 is reminiscent of the bounded max register of [2], where the value is also written bit-by-bit starting from the least significant bit using the same tree structure. Note, however, that their max register implementation is leaking, as writers condition their writes on values read at each level. (For a reader familiar with this algorithm: if the current bit of the value being written is 0, but the bit at this level is 1, the writer terminates its execution, but learns that a larger value has been written. Omitting this check is difficult since it prevents writers of smaller values from overwriting the bits of larger values.)

Algorithm 2 is a fully-non-leaking bounded max register. It uses two arrays of binary registers, $S[0..N-1]$ and $I[0..N-1]$. The value 1 in $S[x]$ marks that x is *set*, while a 1 in $I[x]$ marks that x is *invalidated*. A read operation returns x only if it finds that $I[x] = 0$ and $S[x] = 1$. $S[0]$ is initially 1, representing that the max register is initially 0.

A $\text{WRITEMAX}(x)$ first writes 1 to $S[x]$ then invalidates all markers $I[y]$ with $y < x$ by writing 1 to $I[y]$. This both records that some process wrote at x and eliminates evidence of smaller values. A READMAX operation goes over the arrays of markers I , from $N-1$ downwards. If it finds an index x where $I[x] = 0$ and $S[x] = 1$, it invalidates all $I[y]$ for $y < x$ and returns x . However, if the scan encounters an invalid marker ($I[x] = 1$), it aborts the downward search and initiates an upward scan starting from $x+1$. Upon finding the first index y in this upward scan such that $I[y] = 0$, the operation similarly invalidates all preceding markers and returns y .

Termination of `WRITEMAX` is immediate. If `READMAX` finds a set value during its downward scan, it returns a valid value. If it finds that $I[x] = 1$ during its downward scan, the subsequent upward scan (starting from $I[x + 1]$) is guaranteed to find 0 at some $I[y]$, $y > x$, because the entry in I corresponding to the largest value ever written cannot be invalidated. Since no larger value is ever written, neither this entry nor any entries corresponding to larger values can be invalidated. This is formalized in the next proposition.

► **Proposition 6.** *Let x be the largest value such that $S[x].\text{write}(1)$ was performed during the execution. Then for all $y \geq x$, $I[y] = 0$.*

A `WRITEMAX(x)` operation wop is *visible* if wop changes $S[x]$ from 0 to 1. We first order all visible `WRITEMAX` operations by their arguments, from smallest to largest. Note that since each $S[x]$ can be set to 1 only once, there is at most one visible `WRITEMAX(x)` for each x .

Next, we place the remaining `WRITEMAX` operations and the `READMAX` operations. We go through these operations by the order they are invoked and place each one after a visible `WRITEMAX` operation. Each operation is placed immediately before the next visible `WRITEMAX` operation in the order, or at the end if no such visible operation exists.

To place the `READMAX` operations, we need to show that they return a value written by some `WRITEMAX`. Since invalidations occur in descending order, and $S[x]$ is set before invalidating values smaller than x , we have the following proposition.

► **Proposition 7.** *If $I[x] = 1$ for some x , then at the same point in the execution, either $I[x + 1] = 1$ or $S[x + 1] = 1$.*

Consider a `READMAX` operation rop that returns x . To return x , rop must either directly read $S[x] = 1$, or read $I[x] = 0$ during an upward scan. By Proposition 7, the latter case also implies that $S[x]$ contains 1. This implies that there is a visible `WRITEMAX(x)` operation that sets this entry, and rop is placed after it.

Consider an invisible `WRITEMAX(x)` operation wop . It performs $S[x].\text{write}(1)$ only after $S[x]$ has been previously set. Consider all visible `WRITEMAX` operations that successfully set their entries in S prior to the `write` invoked by wop . We place wop after the visible `WRITEMAX` operation with the maximal argument among them. Note that wop is placed after a visible `WRITEMAX(y)` operation such that $y \geq x$.

This construction respects the sequential specification of a max-register. Because visible `WRITEMAX` are ordered by their arguments and no invisible `WRITEMAX` is placed before a visible `WRITEMAX` with a smaller or equal argument, every `READMAX` returns the maximum value among all `WRITEMAX` operations linearized before it.

The next lemma is the key to proving that the ordering preserves the real-time order between non-overlapping operations. It follows from the fact that `READMAX` operations returning x , and `WRITEMAX(x)` operations invalidate all $I[y]$ for $y < x$.

► **Lemma 8.** *Let op be a `READMAX` operation that returns x or a `WRITEMAX(x)` operation. For any operation op' that follows op , where op' is a `READMAX` operation returning y or a visible `WRITEMAX(y)` operation, it holds that $x \leq y$.*

Since visible `WRITEMAX` operations are ordered by their argument, Lemma 8 proves that the ordering of the visible `WRITEMAX` operations and `READMAX` operations respects the real-time order. The real-time order is naturally preserved between invisible `WRITEMAX` operations, as well as between a visible `WRITEMAX` and an invisible `WRITEMAX`.

It only remains to consider the real-time order between invisible `WRITEMAX` and `READMAX` operations. Consider an invisible `WRITEMAX` operation wop placed after a visible

WRITEMAX(x) operation. Any READMAX operation that precedes wop must return a value smaller than or equal to x . Moreover, by Proposition 6, any READMAX operation that follows wop must return a value greater than or equal to x .

We apply the same methodology as for the single-writer register (Lemma 5) to prove that the max register is fully-non-leaking. We construct a minimal execution that is indistinguishable from the original execution to process p . This minimal execution includes only the WRITEMAX operations necessary to explain the values returned by the READMAX operations of p . The proof for the max register is slightly more involved, since READMAX operations returning x may view different values from memory, unlike the single-writer register, where a READ returning x always views the same unique values.

► **Lemma 9.** *Algorithm 2 is a fully-non-leaking implementation of a bounded max register.*

Proof. Let α, β be executions of respectively the abstract max register and Algorithm 2 such that α is the linearization of β . Let $\alpha' \sim_p \alpha$. We show that there exists an execution $\beta' \sim_p \beta$ containing exactly the operations in $\min_p(\alpha')$. By Lemma 2, this implies that the algorithm is fully-non-leaking.

We construct β' by iterating over the operations of p in $\min_p(\alpha')$, so that β' contains only the operations from $\min_p(\alpha')$ that appear before the operation by p . Since $\min_p(\alpha') \sim_p \alpha$ and α is the linearization of β , both $\min_p(\alpha')$ and β contain the same operations by process p , appearing in the exact same order.

If the next operation of p is a WRITEMAX operation, we simply append it to β' . We can do this because p only writes during the operation and never reads from memory.

If the next operation of p is a READMAX operation returning x , then since $\min_p(\alpha')$ respects the sequential specification, there is a preceding WRITEMAX(x) in $\min_p(\alpha')$, and no preceding WRITEMAX in $\min_p(\alpha')$ has argument $y > x$. By construction, the latter also holds in β' .

We consider the WRITEMAX operation with the maximum argument (if it exists) in β' . If the argument for this WRITEMAX is x , we simply append the READMAX operation by p to β' . Otherwise, there must be some WRITEMAX(x) operation by process $q \neq p$, that precedes the READMAX operation in $\min_p(\alpha')$. We consider two cases.

If p returns x without performing an upward scan. That is, p finds x is not yet invalidated ($I[x] = 0$) and set ($S[x] = 1$). We append a complete WRITEMAX(x) operation by process q to β' , and then append the READMAX operation by p to β' .

Otherwise, p finds no value that is not invalidated and set while scanning downward the arrays I and S , and after seeing some value $y < x$ invalidated, scan the invalidation array upwards. Consider a READMAX returning z or a WRITEMAX(z) by process p that precedes the READMAX operation. It must hold that $z \leq y$. This is because after the operation returns $S[z] = 1$, and in the downward scan p reads 0 from $S[w]$ for any $w > y$. Hence, by construction, any WRITEMAX operation in β' must have an argument smaller than or equal to y . This implies that $I[z] = 0$ and $S[z] = 0$ for any $z > y$.

We let p execute the steps of its READMAX operation, pausing it just before it reads $I[y]$ during its downward scan. Then, we append a complete WRITEMAX(x) operation by process q to β' , and let p complete its remaining READMAX steps.

By construction, $\beta' \sim_p \beta$. In addition, due to the minimality of $\min_p(\alpha')$, any operation not executed by p exists solely to justify a value returned by a READMAX operation by p . Hence, $\min_p(\alpha')$ does not contain any operation that is not included in β' . ◀

5 No Wait-Free Unbounded Fully-Non-Leaking Max Register

The fully-non-leaking implementation of a bounded max register from binary registers (Algorithm 2) can accommodate an unbounded domain by using infinite arrays. Unfortunately, with this change, writing increasing values may prevent a READMAX from finding a valid marker ($I[x] = 0$), and thus, the implementation is only *lock-free*. On the other hand, if we allow base objects with infinite state, there is a wait-free fully-non-leaking unbounded max register from unbounded responseless CAS registers (see Appendix A).

In this section, we prove that compromising one of these properties is inherent: there is no wait-free linearizable fully-non-leaking unbounded max-register, from base objects with a finite number of states. There is no restriction on the *number* of base objects.

The proof considers two processes, a writer w and a reader r . For the lower bound, we use the following property of fully-non-leaking implementations: a process is unaware of the steps of READMAX operations performed by other processes.¹ Hence, assuming an execution where r only performs READMAX operations and w performs only WRITEMAX operations, the writer always performs exactly the same steps regardless of the steps of the reader. More formally, for any execution β in which process w performs only WRITEMAX operations (alongside any number of READMAX operations by the other process), there is an execution β' consisting exclusively of w 's WRITEMAX operations such that $\beta \sim_w \beta'$.

Clearly, as each base object holds only a finite number of states, and WRITEMAX operations can write arbitrarily large values, the number of objects a writer needs to modify is unbounded. Intuitively, what the lower bound shows is that this also causes the reader to access an unbounded number of objects. Therefore, to guarantee that the reader eventually returns, the reader must notify the writer of its presence, and the writer must record a value that the reader is allowed to return. However, because we require the implementation to be non-leaking, the writer is unaware of the reader's request.

The proof considers a restricted set of executions of the following form. The writer, w , performs an infinite sequence of WRITEMAX operations, writing $1, 2, 3, \dots$. Interleaved between these operations are steps by the reader r ; we will show that these steps belong to a single READMAX operation. Such an execution is specified stating the indexes of the WRITEMAX operations after which r takes steps.

Such an execution is determined by a k -sequence, $k \geq 0$, namely, a sequence of k positive integers $\tau = t_1 < t_2 < \dots < t_k$; for $k = 0$, τ is the empty sequence. For a k -sequence $\tau = t_1 < t_2 < \dots < t_k$, $\alpha(\tau)$ is the execution in which the writer w performs an infinite sequence of WRITEMAX(j) operations, and the reader r executes k steps interleaved among these writes, as follows: r takes its i -th step, $1 \leq i \leq k$, after the t_i -th and before the $(t_i + 1)$ -th WRITEMAX operation by w . We abuse terminology by using “point t_i ” to denote the exact moment in the execution immediately before the reader r takes its i -th step.

For $k > 0$, an infinite set of k -sequences is *unlimited* if their first entries are distinct; for $k = 0$, the singleton set containing the empty sequence is unlimited.

A set $\mathcal{T}_{k+1}$ of $(k+1)$ -sequences *extends* a set $\mathcal{T}_k$ of k -sequences if every sequence $\tau \in \mathcal{T}_k$ has exactly one extension $\tau' \in \mathcal{T}_{k+1}$, and every sequence $\tau' \in \mathcal{T}_{k+1}$ is an extension of some sequence $\tau \in \mathcal{T}_k$. (That is, there is a one to one mapping from $\mathcal{T}_k$ onto $\mathcal{T}_{k+1}$, in which each k -sequence is mapped to a unique $(k+1)$ -sequence that extends it.)

¹ This property also holds if the reader never writes. Hence, this lower bound also shows that a reader must write in any wait-free implementation of an unbounded max register from finite base objects.

► **Lemma 10.** *For every $k \geq 0$ there is an unlimited set of k -sequences $\mathcal{T}_k$ such that $\alpha(\tau_1) \sim_r \alpha(\tau_2)$ for every pair $\tau_1, \tau_2 \in \mathcal{T}_k$; furthermore, if $k > 0$, then $\mathcal{T}_k$ extends $\mathcal{T}_{k-1}$.*

Proof. We prove the lemma by induction on k , maintaining the additional property that the set $\mathcal{T}_k$ is also *extendable* in the following sense: there is an infinite set of positive integers I_k such that, for every base object b , there is a state s_b such that for any sequence $\tau \in \mathcal{T}_k$ and any integer $t \in I_k$, if concatenating t to τ , denoted $\tau \cdot t$, yields a valid $(k+1)$ -sequence, then in the execution $\alpha(\tau \cdot t)$, object b is in state s_b at point t . **Base case:** $\mathcal{T}_0$, which contains only the empty tuple, is unlimited by definition. Furthermore, because every base object has only a finite number of states, for any base object b , there is at least one state s_b that b is in infinitely often during the execution $\alpha(\cdot)$. Therefore, the empty sequence $()$ has infinitely many distinct 1-extensions where the object b is in state s_b immediately before r 's first step. This implies that $\mathcal{T}_0$ is extendable.

Inductive step: By the inductive assumption, all executions $\alpha(\tau)$ for $\tau \in \mathcal{T}_k$ are indistinguishable to r . Therefore, in its $(k+1)$ -th step, r accesses the same base object b .

Because $\mathcal{T}_k$ is extendable, choosing any of the guaranteed extensions for each $\tau \in \mathcal{T}_k$ ensures that $\alpha(\tau_1) \sim_r \alpha(\tau_2)$ for all extensions τ_1 and τ_2 . We now show how to pick these extensions so that the extendability condition holds for the base object b .

Let $t, t' \in I_k$ such that $t < t'$. If we extend any sequence $\tau \in \mathcal{T}_k$ by appending t , we know b is in state s_b at point t in $\alpha(\tau \cdot t)$.

Because the implementation is non-leaking, b is in the same state at point t' in $\alpha(\tau \cdot t \cdot t')$ for all $\tau \in \mathcal{T}_k$. This is, because the $(k+1)$ -th step of process r accesses and modifies b in the same way across all $\tau \cdot t$ sequences, and in a non-leaking implementation the writer w performs the same sequence of base operations after point t regardless of the reader's steps.

We color a pair (t, t') by the state of b at point t' . Since the base object b has a finite number of possible states, the number of colors is finite. By the infinite Ramsey theorem, there exists an infinite subset $I_{k+1} \subseteq I_k$ and state s'_b of b such that for every pair $t, t' \in I_{k+1}$ with $t < t'$ and $\tau \in \mathcal{T}_k$, the object b is guaranteed to be in state s'_b at point t' in $\alpha(\tau \cdot t \cdot t')$.

To construct $\mathcal{T}_{k+1}$, consider a sequence $\tau \in \mathcal{T}_k$. There are infinitely many integers in I_{k+1} strictly greater than the k -th entry of τ ; we select such an integer $t \in I_{k+1}$ and append it to τ to obtain a $(k+1)$ -sequence $\tau' = \tau \cdot t$. $\mathcal{T}_{k+1}$ is the set of all these τ' sequences.

Next, we prove extendability for any base object $b' \neq b$. By the inductive hypothesis, b' is in state $s_{b'}$ at point t in $\alpha(\tau \cdot t)$. Let $t' \in I_{k+1}$ such that $t < t'$. Since $I_{k+1} \subseteq I_k$, $t' \in I_k$. Because the implementation is non-leaking, in the execution $\alpha(\tau \cdot t \cdot t')$, the object b' must be in state $s_{b'}$ at point t' . This holds because the writer w performs the same primitive operations regardless of the reader's steps, and r 's $(k+1)$ -th step only accesses b , leaving the state of b' completely unchanged.

For $k = 0$, the set $\mathcal{T}_1$ is formed by an infinite number of distinct 1-extensions of the empty tuple. Because these are distinct 1-sequences, their first entries are strictly distinct, making $\mathcal{T}_1$ unlimited by definition. For $k \geq 1$, since $\mathcal{T}_{k+1}$ extends $\mathcal{T}_k$ and $\mathcal{T}_k$ is unlimited, we have that $\mathcal{T}_{k+1}$ is also unlimited.

Because r 's local view is identical prior to its $(k+1)$ -th step, and it observes the identical state s_b during this step across all executions, $\alpha(\tau_1) \sim_r \alpha(\tau_2)$ for all $\tau_1, \tau_2 \in \mathcal{T}_{k+1}$. ◀

► **Theorem 11.** *There is no wait-free implementation of an unbounded fully-non-leaking max register for two processes using only base objects that each have a finite number of states.*

Proof. We construct an infinite sequence of increasingly longer executions, in each of which the reader r takes infinitely many steps but never returns from its READMAX operation.

By Lemma 10, for every $k \geq 1$, there is an unlimited set of k -sequences $\mathcal{T}_k$ such that $\mathcal{T}_k$ extends $\mathcal{T}_{k-1}$ and $\alpha(\tau_1) \sim_r \alpha(\tau_2)$ for all $\tau_1, \tau_2 \in \mathcal{T}_k$.

Let $k \geq 1$ and $\tau \in \mathcal{T}_k$. Suppose, by way of contradiction, that the reader returns a value v at some point in $\alpha(\tau)$. Then it returns the same value v at some point in each execution $\alpha(\tau')$ for each $\tau' \in \mathcal{T}_k$, as it cannot distinguish these executions.

Because $\mathcal{T}_k$ is unlimited, there is a sequence $\tau' \in \mathcal{T}_k$ whose first entry is $> v$. In the execution $\alpha(\tau')$, the writer w completes a $\text{WRITEMAX}(v+1)$ operation before the reader r takes its first step. Thus, the READMAX operation returning v when a $\text{WRITEMAX}(v+1)$ precedes it violates the linearizability of a max register. Thus, we get increasingly longer executions, in which the reader taking an unbounded number of steps without returning, contradicting wait-freedom. $\blacktriangleleft$

6 Argument-Non-Leaking Wait-Free Stack and Lock-Free Queue

We next turn to argument-non-leaking implementations. We adapt existing implementations of a stack [1] and a queue [18] by applying a common technique from the world of secret-sharing: randomly splitting a value into two parts, each of which gives no information about the value in isolation. To protect the arguments of the stack and the queue from leakage, we write each part into a different location in memory, so that a process does not learn the element unless it reads both locations. This makes correctness somewhat more tricky, as now an element is not stored in one place and cannot be accessed atomically.

6.1 Wait-Free Stack

Our stack is based on the wait-free stack of Afek, Gafni and Morrison [1], reproduced on the left side of Algorithm 3. (In the same paper the authors also present a swap-based stack, which is already argument-non-leaking.) This implementation leaks operations and arguments. For example, suppose that when process p invokes PUSH for the first time and calls $i \leftarrow \text{range.fetch\&add}$, the value fetched is $i = 2$. Then process p learns that some other process invoked PUSH before it, even though the abstract semantics of the stack does not reveal this information. This is *operation leaking* (A_{op}). But the implementation also leaks *values*: suppose that process p 's first access to the stack is to invoke POP , and it reads a non- $\perp$ value y from some entry $\text{items}[i]$, but then fails the $\text{test\&set}$ on $T[i]$, eventually returning a different value $z \neq y$ from some other entry $\text{items}[j]$ where $j < i$. Then process p learns that y was pushed onto the stack by some other process, even though its interaction with the stack should not reveal this information according to the abstract semantics: it should only have learned that the value z was pushed by some other process, and should not have learned about y . To address this, in our modified stack, instead of writing input x in a single register $\text{items}[i]$, a PUSH operation (Algorithm 3 (right)) randomly splits x into two parts x_1 and x_2 whose sum is x . These parts are written into two distinct registers $R[i]$ and $S[i]$. Crucially, $S[i]$ is written first.

A POP operation op trying to obtain the i -th value pushed into the stack first checks that the corresponding PUSH operation has terminated by reading $R[i]$. If $R[i] \neq \perp$, the process tries to claim slot i by performing a $\text{test\&set}$ on $T[i]$. Only if this slot is successfully claimed does the POP operation read $S[i]$, retrieve x , and return it. If x has already been returned by another POP , then the $\text{test\&set}$ on $T[i]$ fails, and op cannot be sure what is the input of the i -th push operation. In this case the POP operation gives up on obtaining the i -th value and attempts to obtain the $(i-1)$ -th value instead. This continues until either a value is successfully obtained, or the operation reaches $i = 0$ and returns the special value EMPTY .

■ **Algorithm 3** Afek, Gafni and Morrison’s wait-free stack (left) and its non-leaking variant (right)

1: Shared objects	1: Shared objects
2: <i>range</i> : fetch&add object, initially 1	2: <i>range</i> : fetch&add object, initially 1
3: <i>items</i> : array $[1, \dots]$ of registers; initially $\perp$	3: <i>R</i> : array $[1, \dots]$ of registers; initially $\perp$
4:	4: <i>S</i> : array $[1, \dots]$ of registers; initially $\perp$
5: <i>T</i> : array $[1, \dots]$ of test&set objects; initially $\perp$	5: <i>T</i> : array $[1, \dots]$ of test&set objects; initially $\perp$
6: function PUSH(<i>x</i>)	6: function PUSH(<i>x</i>)
7: <i>i</i> $\leftarrow$ <i>range</i> .fetch&add(1)	7: <i>i</i> $\leftarrow$ <i>range</i> .fetch&add(1)
8:	8: <i>N</i> $\leftarrow$ random number
9: <i>items</i> [<i>i</i>].write(<i>x</i>)	9: <i>S</i> [<i>i</i>].write(<i>N</i>)
10:	10: <i>R</i> [<i>i</i>].write(<i>x</i> + <i>N</i>)
11: function POP()	11: function POP()
12: <i>t</i> $\leftarrow$ <i>range</i> .fetch&add(0) − 1	12: <i>t</i> $\leftarrow$ <i>range</i> .fetch&add(0) − 1
13: for <i>i</i> from <i>t</i> downto 1 do	13: for <i>i</i> from <i>t</i> downto 1 do
14: <i>y</i> $\leftarrow$ <i>items</i> [<i>i</i>].read()	14: <i>y</i> $\leftarrow$ <i>R</i> [<i>i</i>].read()
15: if <i>y</i> $\neq \perp$ then	15: if <i>y</i> $\neq \perp$ then
16: if <i>T</i> [<i>i</i>].test&set() then	16: if <i>T</i> [<i>i</i>].test&set() then
17:	17: <i>N</i> = <i>S</i> [<i>i</i>].read()
18: return <i>y</i>	18: return <i>y</i> − <i>N</i>
19: return EMPTY	19: return EMPTY

We remark that although an operation does not learn the arguments or return values of other operations, it can sometimes infer their existence: for example, a POP that reads the value *t* from *range* learns that at least *t* − 1 PUSH operations have started. Similarly, a failed test&set on an entry in *T* leaks the existence of a POP operation.

Our proof for the linearizability of Algorithm 3 follows [1]. In particular, we reuse their method of constructing a sequential history *after* the complete concurrent history is known, rather than assigning explicit linearization points to operations.

Given a finite execution β , they construct a *sequential* history $Seq(\beta)$ by going over the events of β , so that $Seq(\beta)$: (i) satisfies the stack specification, (ii) respects the real-time order of non-overlapping operations in β , and (iii) contains all operations completed in β .

We reuse this construction verbatim, with minor adaptations to account for the different per-slot claiming mechanism and the encoding of values used in our algorithm.

Each PUSH operation obtains a unique index *i* by executing *range*.fetch&add(1). The pushed value becomes visible to POP operations when a non-NULL value is written to *R*[*i*]. As in AGM, indices totally order PUSH operations and represent abstract stack positions.

In both AGM’s and our stack, a POP operation claims slot *i* by winning a test&set on that slot. Thus, Algorithm 3 follows the same structure of the proof: in both algorithms, each slot can be claimed by at most one POP.

► **Lemma 12.** *For every index *i*, at most one POP operation in β returns a value from slot *i*.*

Algorithm 3 (right) encodes each pushed value *x* by writing a random value *N* to *S*[*i*] and writing *x* + *N* to *R*[*i*]. A POP that successfully claims slot *i* retrieves *S*[*i*] and *R*[*i*], and returns their difference.

► **Lemma 13.** *If a POP operation returns from slot *i*, then it returns exactly the value written by the unique PUSH operation that published slot *i*.*

We now construct the sequential history $Seq(\beta)$ exactly as in AGM. Briefly, $Seq(\beta)$ is built by scanning the events of β and maintaining an auxiliary abstract stack. When a

PUSH publishes slot i , the corresponding PUSH operation is appended to $Seq(\beta)$. When a POP successfully claims slot i , the corresponding POP operation is appended to $Seq(\beta)$ and matched with the most recent unmatched PUSH at index i . A POP that finds no claimable slot is appended as an empty pop.

By Lemmas 12 and 13, this construction is well-defined and mirrors the AGM construction. As shown by AGM, the resulting sequential history $Seq(\beta)$ is a legal stack history: each POP returns the value of the most recent unmatched PUSH, or EMPTY if none exists. Moreover, the construction respects the real-time order of non-overlapping operations, since a POP can only be matched with a PUSH whose publication precedes the claim.

Since the construction of $Seq(\beta)$ and the associated correctness arguments are identical to those of AGM (using the two lemmas above), it follows that Algorithm 3 (right) implements a linearizable stack.

Finally, we argue that Algorithm 3 does not leak the arguments or return values of operations.

► **Lemma 14.** *Algorithm 3 (right) is argument-non-leaking.*

Proof. Let X be the domain of values that can be pushed into the stack. We begin by observing that for any execution β , process p and set of values $X' \subsetneq X$, if p does not itself push or pop a value from X' in β , then there is an indistinguishable execution $\beta' \sim_p \beta$ where *no process* pushes or pops any value from X' . This is because for any value $x \in X'$ that is not returned from a POP operation by process p , throughout β , process p never reads $R[i], S[i]$ such that $R[i] - S[i] = x$ for any i (process p only ever performs a successful **test&set** on $T[i]$ inside a POP operation, immediately prior to returning $R[i] - S[i]$). Thus, we can construct β' from β as follows: first, choose some value $y \in X \setminus X'$ arbitrarily (such a value exists since $X' \subsetneq X$). In any **PUSH**(x) operation where $x \in X'$, we replace x by y , and inside the operation we replace the value of N by $N' = x - y + N$, so that the value written into $R[i]$ is the same in β, β' (as $y + N' = x + N$). This implies that whenever process p reads $R[i]$ it will read the same values in β, β' . As we said above, process p never reads $S[i]$ unless it is about to return $R[i] - S[i]$ from a POP, and therefore it does not observe the different value N' that is written into $S[i]$ (as we assumed that process p does not return $y \in X'$ from a POP). Finally, for POP operations we do not need to make any further changes, as any non-EMPTY value returned by POP was previously PUSHed, and in β' no value from X' is PUSHed.

We now prove that the algorithm is argument-non-leaking. Since PUSH operations do not have a return value and POP operations do not take an argument, it is convenient to denote the atomic propositions for these operations by $op(m, V)$ where $m \in \{\text{PUSH}, \text{POP}\}$, and $V \subseteq P \times X$. The proposition $op(\text{PUSH}, V)$ corresponds to a PUSH operation performed by a process q of a value x such that $(q, x) \in V$, and the proposition $op(\text{POP}, V)$ corresponds to a POP operation performed by a process q that returns a value x such that $(q, x) \in V$.

Let β be a concrete execution, let $\alpha = r(\beta)$ where r is the linearization mapping defined above (it is not important for our purposes here; any legal mapping would do), and let $V \subseteq P \times X$ be a set of pairs of process ID and element such that $\alpha \not\models K_p op(m, V)$ for $m \in \{\text{PUSH}, \text{POP}\}$. Then there is an indistinguishable abstract execution $\alpha' \sim_p \alpha$ where $\alpha' \not\models op(m, V)$, that is, in α' , no operation $op(m, q, x)$ such that $(q, x) \in V$ appears. In particular this implies that in α' , process p itself does not invoke $op(m, q, v)$ for any value x such that $(p, x) \in V$, and since $\alpha' \sim_p \alpha$, the same holds in α . Finally, since $\alpha = r(\beta)$, the same holds in β . By our claim above, there is an indistinguishable execution $\beta' \sim_p \beta$ where for any $(q, x) \in V$, process q does not invoke **PUSH**(x) or return the value x from POP. This means $\beta' \not\models op(m, V)$, and since $\beta' \sim_p \beta$, we have $\beta \not\models K_p op(m, V)$, as desired. ◀

■ **Algorithm 4** Non-leaking lock-free queue, adapted from Li [18]

```

1: Shared objects
2:   range: fetch&add object, initialized to 1
3:   R: array [1, ...] of registers (each initially NULL)
4:   S: array [1, ...] of registers (each initially NULL)
5:   T: array [1, ...] of test&set objects (each initially NULL)
6: function ENQUEUE(x)
7:   t ← range.fetch&add(1)
8:   N ← random number
9:   S[t].write(N)
10:  R[t].write(x + N)
11: function DEQUEUE( )
12:  t ← 0; emptied ← 0
13:  repeat
14:    told ← t
15:    t ← range.fetch&add(0) − 1
16:    emptiedold ← emptied; emptied ← 0
17:    for i from 1 upto t do
18:      y ← R[i].read()
19:      if y ≠ NULL then
20:        if T[i].test&set() then
21:          N = S[i].read()
22:          return y − N
23:      emptied ← emptied + 1
24:  until t = told and emptied = emptiedold
25:  return EMPTY

```

6.2 Lock-free Queue

Li's queue [18] is similar to the AGM stack; the only difference is that enqueues add elements to the tail, while dequeues remove elements from the head. Like the AGM stack, it leaks ENQUEUE operations. However, we can make it non-leaking with the same technique of randomly splitting each enqueued value x into two parts whose sum is x .

Our non-leaking queue is presented in Algorithm 4. The DEQUEUE operation reads the current value of *range* and iterates over the elements from the first one down to the tail. Unlike a POP, it cannot simply return EMPTY if it does not find an element to return. It can only do so if it observes that no new elements have been enqueued since its last traversal. Otherwise, it rereads the current value of the tail and restarts the traversal.

As in the AGM stack, both enqueues and dequeues claim slots in the infinite array. By using arguments similar to the proof of the stack algorithm we can follow the proof of the original algorithm and show that it is linearizable and lock-free. The argument-non-leaking property is also similar: for each value x that is not enqueued or dequeued by a process p , we can construct an execution that is indistinguishable to p , where no process enqueues or dequeues x . This allows us to argue that for any concrete execution β where the abstract specification does not allow p to learn some proposition $op(m, V)$, there is an indistinguishable concrete execution $\beta' \sim_p \beta$ where $op(m, V)$ in fact does not hold, and therefore p does not learn $op(m, V)$ in β .

7 Input-Non-Leaking Wait-Free Approximate Agreement

Finally, we extend our framework to capture information leaking in the context of decision tasks, using approximate agreement as an example. Algorithms for approximate agreement are often based on successive halving phases, in each of which, processes write their current proposed values to the shared memory, collect all the values, and take their mean or median [6]. This structure clearly allows processes to know the input values of other processes, even when they end up deciding on their own input. We show, however, that a slight adaptation of Schenk's approximate agreement algorithm [22] does not leak inputs.

7.1 Defining Input-Non-Leaking Approximate Agreement

The framework of Section 2 can be used to define non-leaking *decision tasks*. For simplicity, we concentrate on colorless tasks, and approximate agreement, in particular.

A *colorless* task T is a triplet $(\Delta, \mathcal{I}, \mathcal{O})$, where Δ maps each set of inputs in $\mathcal{I}$ to a set of allowed outputs in $2^{\mathcal{O}}$, subject to for any $\sigma \subset \sigma' \in \mathcal{I}$, $\Delta(\sigma) \subseteq \Delta(\sigma')$. Equivalently, T may be specified as an object $O_T = (Q, \Sigma, s_0, \delta)$ with a single method (hence omitted). Intuitively, each state $q \in Q$ records inputs received and outputs returned, so that future return actions output values allowed by Δ , and consistent with past outputs. Formally, $\Sigma = \text{Call} \cup \text{Ret}$, $Q \subseteq \mathcal{I} \times \mathcal{O}$ with $s_0 = (\emptyset, \emptyset)$. For a call action $\text{Call}(v, _)$, $(q, \text{Call}(v, _), q') \in \delta$ if and only if $q = (I, O)$, $q' = (I', O')$ with $I' = I \cup \{v\}$ and $O = O'$. A value $v \in \mathcal{O}$ may be returned given state $q = (I, O)$ if this is allowed by the task relation Δ . That is, $(q, \text{Ret}(v, _), q') \in \delta$ if and only if $q = (I, O)$, $q' = (I', O')$ with $I' = I$, $O = O' \cup \{v\}$ and $O' \in \Delta(I')$. Note that $O_T = (Q, \Sigma, s_0, \delta)$ is a deterministic LTS.

Therefore, an algorithm C solves T if C refines O_T . Refinements suitable for objects defined from tasks have been identified in [9, 21].

An algorithm L solving a colorless task $T = (\Delta, \mathcal{I}, \mathcal{O})$, for a set process P , is *input-non-leaking* if L is A_{input} -non-leaking, where A_{input} is the following set of atomic propositions:

$$A_{\text{input}} = \{ \exists p \in P \exists v \in \mathcal{O} . \text{op}(p, u, v) \quad : \quad u \in \mathcal{I} \},$$

and $\text{op}(p, u, v)$ holds in an execution γ if process p has input u and outputs v .

In *approximate agreement*, each process p_i has an input x_i in some interval X and is required to output a value y_i with $\min(X) \leq y_i \leq \max(X)$ and such that all outputs are within distance ϵ of one another, where $\epsilon > 0$.

We can fix the input range to be $[0, 1]$ and $\epsilon = 1/N$ for some integer N . A fixed input range $[a, b]$ and ϵ can be handled by scaling the inputs (i.e., $\bar{x} = (x - a)/(b - a)$) so that they are in the interval $[0, 1]$, setting $\bar{\epsilon} = \lceil \epsilon/(b - a) \rceil$, and returning a scaled output (i.e., $y = \bar{y}(b - a) + a$). An approximate agreement algorithm is input-non-leaking if it is A_{input} -non-leaking, for $A_{\text{input}} = \{ \exists p \in P \exists y \in [0, 1] . \text{op}(p, x, y) \quad : \quad x \in [0, 1] \}$.

7.2 Schenk's Algorithm (Slightly Modified)

At the heart of Schenk's approximate agreement algorithm for known initial range [22] are *halving objects* (Algorithm 5). Given inputs drawn from a known interval $I = [a, b]$, a halving object H_I with parameter I allows processes to get outputs that are in the range of the inputs, and in one half of I . In other words, it is an implementation of approximate agreement with input range I and $\epsilon = |I|/2$, with the additional *partition property* that outputs are either all in the lower half of I , or all in the upper half of I . The algorithm uses two binary registers R_0 and R_1 , which signal the existence of an input in the lower and upper half of

■ **Algorithm 5** Halving Object $H_{[a,b]}$

```

1: Shared variables  $R_0, R_1$  : binary multi-writer registers, initially false.
2: function HALVE( $x$ )  $\triangleright x \in [a, b]$ 
3:   if  $x \leq (a + b)/2$  then  $k \leftarrow 0$  else  $k \leftarrow 1$ 
4:    $R_k.write(true)$ 
5:   if  $x = (a + b)/2$  then return  $x$ 
6:   if  $R_{1-k}.read()$  then return  $(a + b)/2$  else return  $x$ 

```

■ **Algorithm 6** Schenk's approximate agreement, $\epsilon = 1/2^N$ for some integer N

```

1: Shared variables For each interval  $S_{k,\ell} = [k/2^\ell, (k+1)/2^\ell]$ :  $0 \leq \ell \leq N, 0 \leq k < 2^\ell$ : halving
   object  $H_{S_{k,\ell}}$ 
2: function AA $_\epsilon(x)$   $\triangleright x \in [0, 1]$ 
3:    $y \leftarrow x$ 
4:   for  $\ell = 0, \dots, N - 1$  do
5:     let  $k = \min\{k' : x \in [k'/2^\ell, (k'+1)/2^\ell]\}$ ;  $d \leftarrow H_{[k/2^\ell, (k+1)/2^\ell]}.halve(y)$ 
6:     if  $d \neq 0$  and  $d = y = (k+1)/2^\ell$  and  $k+1 < 2^\ell$  then  $y \leftarrow H_{[(k+1)/2^\ell, (k+2)/2^\ell]}.halve(y)$ 
7:     else  $y \leftarrow d$ 
8:   return  $y$ 

```

I respectively. A process p sets one of the registers to *true*, according to which half of I contains its input x . It then reads the other register, and returns x if it is not set (indicating the absence of input in the half not containing its input), and the midpoint of I otherwise.

Line 5 (marked in blue), *was added to the original code, and it does not affect correctness*, as any halve operation with input $x = (a + b)/2$ returns x after writing to the register corresponding to the lower half of $[a, b]$. However, it prevents the invoking process from learning the existence of other operations.

► **Lemma 15** ([22]). *Algorithm 5 with parameter $I = [a, b]$ implements approximate agreement with input range $[a, b]$ and $\epsilon = (b - a)/2$ and ensures the partition property: In every execution, either all outputs are contained in $[a, (a + b)/2]$ or they are all contained in $[(a + b)/2, b]$.*

Since writing to register R_k , $k \in \{0, 1\}$, only reveals which half of I p 's input lies in, Algorithm 5 is input-non-leaking.

► **Lemma 16.** *Let β be an execution of Algorithm 5 with parameter $I = [a, b]$ in which process p 's input is x . For every $x' \in [a, b]$, there is $\beta' \sim_p \beta$ in which no process has input x' .*

Proof. Suppose, without loss of generality, that x lies in the lower half of I . If p reads false from R_1 , β is indistinguishable for p from a solo execution. Otherwise, as R_1 is set to *true* by any process with input $> (a + b)/2$, the input of every process but p can be replaced by any value $x'' \neq x' > (a + b)/2$ without p noticing it. ◀

Schenk's algorithm (Algorithm 6) proceeds in *phases*. Each process p maintains a preference y_p , initially its input. Each phase aims at halving the range of processes' preferences, using halving objects, until they are all contained in an interval of size $\leq \epsilon$. For clarity, assume $\epsilon = 1/2^N$ where N is an integer. In each phase ℓ , $0 \leq \ell \leq N - 1$, the input range $[0, 1]$ is subdivided into intervals of the form $S_{k,\ell} = [k/2^\ell, (k+1)/2^\ell]$, for an integer k , $0 \leq k < 2^\ell$, each with a designated halving object $H_{S_{k,\ell}}$. The following is maintained: All preferences at the beginning of phase ℓ are contained in a common sub-interval $S_{c,\ell}$.

Reducing the range of preferences and maintaining the invariant is easy if every process is able to identify this common sub-interval $S_{c,\ell}$. By invoking $H_{S_{c,\ell}}.halve(y_p)$, each process p

obtains a new preference. By the partition property, they all lie in the same sub-interval $S_{2c,\ell+1}$ or $S_{2c+1,\ell+1}$.

For process p , if y_p is strictly contained in $S_{k,\ell}$, c can be directly inferred from y_p . The main difficulty is when the preference y_p of a process p has the form $k/2^\ell$, which is in at the boundary of two sub-intervals $S_{k-1,\ell}$ and $S_{k,\ell}$. In that case, p invokes both halving objects $H_{S_{k-1,\ell}}$ and $H_{S_{k,\ell}}$, in that order. It is observed in [22] that only one of the two objects associated with these intervals may return a value $\neq k/2^\ell$. Indeed, if at the beginning of the phase, all preferences are contained in one of the two intervals, for example $S_{k,\ell}$, then every invocation of $H_{S_{k-1,\ell}}$ has the same input $k/2^\ell$ and thus returns this value.

7.3 Proof of Correctness and Non-Leakage

Let β be an execution of Algorithm 6, and let $x_{\min}$ and $x_{\max}$ be the smallest and largest inputs in β . For every iteration ℓ , $0 \leq \ell \leq N-1$ of the for loop, and each process p , y_p^ℓ is the preference of p at the beginning of the iteration, and y_p^N is the value returned by process p . Correctness follows from Lemma 1 of [22]. The proof of this lemma includes the next claim, which we use to prove that the algorithm is input-non-leaking.

► **Lemma 17** (Lemma 1 in [22]). *For all ℓ , $0 \leq \ell \leq N$, there is an integer c_ℓ , $0 \leq c_\ell < 2^\ell$ such that for all processes p , $y_p^\ell \in [c_\ell/2^\ell, (c_\ell+1)/2^\ell] \cap [x_{\min}, x_{\max}]$.*

► **Lemma 18** (Claim in the proof of Lemma 1 in [22]). *For all ℓ , $0 \leq \ell < N$, there is an integer c_ℓ , $0 \leq c_\ell < 2^\ell$, such that for all processes p , $y_p^{\ell+1}$ is the value returned to p by applying $\text{halve}(y_p^\ell)$ to the object $H_{[c_\ell/2^\ell, (c_\ell+1)/2^\ell]}$.*

► **Lemma 19.** *Let x be the input of p in β . For every $x' \neq x$, there is an execution $\beta' \sim_p \beta$ in which no process has input x' .*

Proof. For each ℓ , $0 \leq \ell < N$, let S^ℓ be the interval $[c_\ell/2^\ell, (c_\ell+1)/2^\ell]$ defined in Lemma 18. Denote by S^N the half of S^{N-1} to which all final preferences belong to (by the partition property of $H_{S^{N-1}}$, all y_p^N lie in the same half of S^{N-1}).

As any process p invokes halve on object H_{S^ℓ} only if $y_p^\ell \in S^\ell$, $S^0 \supset \dots \supset S^{N-1} \supset S^N$.

For the construction of β' , we use $N+1$ processes $q_0, \dots, q_N$ with input $x_0, \dots, x_N$ respectively. For each ℓ , $0 \leq \ell < N$, we choose $x_\ell \neq x'$ in the interior of $S^\ell \setminus S^{\ell+1}$, and $x_N \neq x'$ in the interior of S^N . Starting from the initial configuration, we let q_N run until it returns, and then run each process q_i , $0 \leq i < N$ until it is about to apply halve on H_{S^i} . Denote by β'_0 this execution. Note that in its solo run, q_N applies $\text{halve}(x_N)$ to each object $H_{S^0}, \dots, H_{S^{N-1}}$ (as it is solo), and does not access any other halving objects (as x_N is not at the boundary of any sub-interval S_k^ℓ). Process q_ℓ , where $\ell < N$, behaves similarly as q_N in any iteration $\ell' < \ell$, getting back x_ℓ from its call to $H_{S^{\ell'}}.\text{halve}()$. Indeed, for every $\ell' < \ell$, x_ℓ and x_N lie in the same half of $S^{\ell'}$. q_ℓ thus writes to the same register of $H_{S^{\ell'}}$ as q_N , and, as q_N , see the other register in its initial state. This is because every process q_j with input x_j in the other half of $S^{\ell'}$ stops running before accessing $H_{S^{\ell'}}$.

We then extend β'_0 by adding steps of p and possibly one more step by q_i , $0 \leq i < N$. Assume inductively that we have constructed prefix $\beta'_{\ell-1}$ and that it is indistinguishable for p from β up to the beginning of iteration ℓ of the for loop.

If p invokes $H_S.\text{halve}(y)$ with $S \neq S^\ell$ in its ℓ th iteration of the for loop in β , the invocation returns y . Otherwise, p would not pass the test of line 6, and would update its preference y with the value returned. By Lemma 18, this would imply that $S = S^\ell$. Since no process q_i , $0 \leq i \leq N$ modifies H_S , p 's invocation $H_S.\text{halve}(y)$ also returns y in β' .

By Lemma 18, every value returned by an invocation of $H_{S^\ell}.\text{halve}()$ lies in the same half (which is $S^{\ell+1}$) of S^ℓ , and this half contains $x_N, \dots, x_{\ell+1}$. Suppose without loss of generality that $S^{\ell+1}$ is the upper half of S^ℓ . In the following, R_0 and R_1 refer to the registers in the implementation of H_{S^ℓ} . Before p , only $q_N, \dots, q_{\ell+1}$ call $H_{S^\ell}.\text{halve}()$, each call with an input in $S^{\ell+1}$. Each of this call therefore writes to R_1 and does not modify R_0 . Hence $R_1 = \text{true}$ and $R_0 = \text{false}$ at the end of $\beta'_{\ell-1}$. Let m be the midpoint of S^ℓ . We consider three cases, depending on which half of S^ℓ p 's preference is.

- $y_p^\ell < m$. p 's invocation of $H_{S^\ell}.\text{halve}$ in β returns a value $\geq m$ (actually m). Hence, after writing to R_0 , p reads $R_1 = \text{true}$ in β . We let p perform iteration ℓ after $\beta'_{\ell-1}$. As $R_1 = \text{true}$ at the end of $\beta'_{\ell-1}$, p takes the same step as in β .
- $y_p^\ell = m$. In this case, according to the implementation of halving objects (Algorithm 5, Line 5) p writes to R_0 and does not read register R_1 in β . We can therefore extend β'_ℓ with the steps taken by p in its iteration ℓ in β .
- $y_p^\ell > m$. In β , p writes to R_1 before reading R_0 . If it reads $R_0 = \text{false}$ in β , which is also the value of R_0 at the end of $\beta'_{\ell-1}$, we let p perform iteration ℓ after β'_ℓ . If p reads true from R_0 in β , we first have q_ℓ perform its next step. Recall that q_ℓ is about to perform $H_{S^\ell}.\text{halve}(x_\ell)$ with $x_\ell \in S_\ell \setminus S_{\ell+1}$. Therefore, x_ℓ lies in the lower half of S_ℓ and the next step of q_ℓ is writing true to R_0 . We then let p perform its ℓ th iteration. ◀

Let aop be an invocation by process p with input x that returns v , and aop' be another invocation by process p' with input x' . By Lemma 19 there is an execution $\beta' \sim_p \beta$ in which aop' does not appear. As the algorithm is wait-free, and solves approximate agreement (by Lemma 15), we have:

► **Theorem 20.** *Algorithm 6 together with Algorithm 5 are a wait-free, $A_{\text{input-non-leaking}}$ implementation of approximate agreement with a known range, using only registers.*

The step complexity is $O(\log(1/\epsilon))$, and the space complexity is $O(\log^2(1/\epsilon))$. As observed by Schenk, space can be reduced to $O(\log(1/\epsilon))$ by dynamically assigning sub-intervals to halving objects, as only at most two such objects are used in each layer.

8 Conclusions

We introduced a framework for reasoning about information exposure in concurrent objects, using epistemic logic in labeled transition systems to compare what a process may know under an abstract specification with what it may know in a concrete implementation. A key novelty in our framework is the use of the abstract object itself as the reference for permissible information exposure. Epistemic reasoning allows us to easily express different levels of information-hiding guarantees, while the use of labeled transition systems smoothly extends the framework to decision tasks.

We showed that strong information-hiding guarantees are achievable for several concurrent objects while preserving linearizability and wait-freedom. At the same time, our impossibility for unbounded max registers shows that the strongest form of information hiding is not always achievable without compromise. Together, these results illustrate both the possibilities and the limitations of preserving information-hiding guarantees in concurrent objects.

Our work leaves open the design of additional non-leaking concurrent objects and their complexity, as well as other general approaches for doing so (like our value splitting technique), and the characterization of which concurrent objects admit non-leaking implementations. It would also be interesting to better understand how information-hiding guarantees can be preserved across layers of abstraction.

References

- 1 Yehuda Afek, Eli Gafni, and Adam Morrison. Common2 extended to stacks and unbounded concurrency. *Distributed Computing*, 20(4):239–252, 2007. doi:10.1007/s00446-007-0023-3.
- 2 James Aspnes, Hagit Attiya, and Keren Censor-Hillel. Polylogarithmic concurrent data structures from monotone circuits. *J. ACM*, 59(1), March 2012. doi:10.1145/2108242.2108244.
- 3 Hagit Attiya, Antonio Fernandez Anta, Alessia Milani, Alexandre Rapetti, and Corentin Travers. Auditing without leaks despite curiosity. In *Proceedings of the ACM Symposium on Principles of Distributed Computing*, PODC '25, page 455–465, New York, NY, USA, 2025. Association for Computing Machinery. doi:10.1145/3732772.3733516.
- 4 Hagit Attiya, Michael A. Bender, Martín Farach-Colton, Rotem Oshman, and Noa Schiller. History-independent concurrent objects. In *Proceedings of the 43rd ACM Symposium on Principles of Distributed Computing*, PODC '24, page 14–24, New York, NY, USA, 2024. Association for Computing Machinery. doi:10.1145/3662158.3662814.
- 5 Hagit Attiya, Michael A. Bender, Martín Farach-Colton, Rotem Oshman, and Noa Schiller. History-independent concurrent hash tables. In *Proceedings of the 57th Annual ACM Symposium on Theory of Computing*, STOC '25, page 1283–1294, New York, NY, USA, 2025. Association for Computing Machinery. doi:10.1145/3717823.3718283.
- 6 Hagit Attiya, Nancy Lynch, and Nir Shavit. Are wait-free algorithms fast? *Journal of the ACM*, 41(4):725–763, July 1994. doi:10.1145/179812.179902.
- 7 Hagit Attiya and Sergio Rajsbaum. Indistinguishability. *Communications of the ACM*, 63(5):90–99, April 2020. doi:10.1145/3376902.
- 8 Hagit Attiya and Jennifer Welch. *Distributed Computing: Fundamentals, Simulations, and Advanced Topics*. Wiley, 2nd edition, 2004. doi:10.1002/0471478210.
- 9 Armando Castañeda, Sergio Rajsbaum, and Michel Raynal. Unifying concurrent objects and distributed tasks: Interval-linearizability. *J. ACM*, 65(6):45:1–45:42, 2018. doi:10.1145/3266457.
- 10 Gregory V. Chockler, Rachid Guerraoui, and Idit Keidar. Amnesic distributed storage. In Andrzej Pelc, editor, *21st International Symposium on Distributed Computing (DISC)*, volume 4731 of *Lecture Notes in Computer Science*, pages 139–151. Springer, 2007. doi:10.1007/978-3-540-75142-7_13.
- 11 Cynthia Dwork and Aaron Roth. *The Algorithmic Foundations of Differential Privacy*. Now Publishers, 2014. doi:10.1561/04000000042.
- 12 Ronald Fagin, Joseph Y Halpern, Yoram Moses, and Moshe Vardi. *Reasoning about knowledge*. MIT press, 2004. doi:10.7551/mitpress/5803.001.0001.
- 13 Joseph A. Goguen and José Meseguer. Security policies and security models. In *IEEE Symposium on Security and Privacy*, pages 11–20. IEEE Computer Society, 1982. doi:10.1109/SP.1982.10014.
- 14 Wojciech M. Golab, Lisa Higham, and Philipp Woelfel. Linearizable implementations do not suffice for randomized distributed computation. In *Proceedings of the 43rd ACM Symposium on Theory of Computing (STOC)*, pages 373–382. ACM, 2011. doi:10.1145/1993636.1993687.
- 15 Maurice Herlihy and Jeannette M. Wing. Linearizability: A correctness condition for concurrent objects. *ACM Trans. Program. Lang. Syst.*, 12(3):463–492, 1990. doi:10.1145/78969.78972.
- 16 Robert M. Keller. Formal verification of parallel programs. *Commun. ACM*, 19(7):371–384, 1976. doi:10.1145/360248.360251.
- 17 Leslie Lamport. On interprocess communication. *Distributed Computing*, 1(2):77–101, 1986. doi:10.1007/BF01786227.
- 18 Zongpeng Li. Non-blocking implementations of queues in asynchronous distributed shared-memory systems. Master's thesis, University of Toronto, 2001. URL: <http://www.collectionscanada.ca/obj/s4/f2/dsk3/ftp05/MQ62967.pdf>.
- 19 Robin Milner. *Communicating and mobile systems - the Pi-calculus*. Cambridge University Press, 1999.

- 20 Moni Naor and Vanessa Teague. Anti-presistence: history independent data structures. In *Proceedings on 33rd Annual ACM Symposium on Theory of Computing (STOC)*, pages 492–501. ACM, 2001. doi:10.1145/380752.380844.
- 21 Gil Neiger. Set-linearizability. In *Proceedings of the Thirteenth Annual ACM Symposium on Principles of Distributed Computing (PODC)*, page 396. ACM, 1994. doi:10.1145/197917.198176.
- 22 Eric Schenk. Faster approximate agreement with multi-writer registers. In *36th Annual Symposium on Foundations of Computer Science, FOCS*, pages 714–723. IEEE Computer Society, 1995. doi:10.1109/SFCS.1995.492673.
- 23 Andrew Chi-Chih Yao. Protocols for secure computations (extended abstract). In *23rd Annual Symposium on Foundations of Computer Science (FOCS)*, pages 160–164. IEEE Computer Society, 1982. doi:10.1109/SFCS.1982.38.

A

 Wait-Free Unbounded Max Register from Responseless CAS

The algorithm utilizes a *responseless* CAS register R (i.e., a CAS that does not return *success* or *failure*). To write a new value x , a process simply executes $R.CAS(y, x)$ for each $y = 0, \dots, x - 1$ (in this order). To read the max-register, a process reads R and returns its value. The code of the algorithm appears in Algorithm 7.

We prove that the algorithm is linearizable by assigning a linearization point to each operation. We say that a $R.CAS(y, x)$ in the execution is *successful* if it changes the value of R from y to x . Let wop be a $WRITE_{MAX}$ operation. If wop performs a successful CAS, we call the operation *visible* and assign its linearization point to this successful step. Otherwise, its linearization point is assigned to the last CAS executed in the loop, specifically the failed $R.CAS(x, x - 1)$ where x is the argument passed to wop . For a $READ_{MAX}$ operation rop , the linearization point is exactly its single execution step, which reads R .

Because each operation is linearized at a point between its invocation and response, this linearization trivially respects the real-time order. Furthermore, since a visible $WRITE_{MAX}$ operation replaces a smaller value previously written by a different visible $WRITE_{MAX}$ operation (or the initial value), the ordering of the visible $WRITE_{MAX}$ operations and $READ_{MAX}$ operations clearly satisfies the sequential specification.

It remains only to show that linearizing the invisible $WRITE_{MAX}$ operations also satisfies the sequential specification. This is established by the following proposition, which shows that a visible $WRITE_{MAX}$ operation with an argument greater than or equal to the argument of the invisible $WRITE_{MAX}$ must have been linearized beforehand. It follows from the fact that R 's value is strictly increasing and that $WRITE_{MAX}$ operations attempt to update R in increasing order.

► **Proposition 21.** *If a $WRITE_{MAX}(x)$ operation performs an unsuccessful $R.CAS(y, x)$ for $y < x$, then at that exact point, the value of R must be strictly greater than y .*

Finally, we prove that the algorithm is fully non-leaking:

► **Lemma 22.** *Algorithm 7 is a fully-non-leaking implementation of an unbounded max register.*

Proof. Let α, β be executions of respectively the abstract register and Algorithm 7 such that α is the linearization of β , as defined above. Let $\alpha' \sim_p \alpha$. We show that there exists a concrete execution $\beta' \sim_p \beta$ that contains exactly the operations in $\min_p(\alpha')$. By Lemma 2, this implies that the algorithm is fully-non-leaking.

Algorithm 7 Fully-non-leaking wait-free unbounded max register from responseless CAS

```

1: Shared objects
2:    $R$ : responseless CAS object, initialized to 0
3: function WRITEMAX( $x$ )                6: function READMAX( )
4:   for  $y$  from 0 upto  $x - 1$  do        7:   return  $R.read()$ 
5:      $R.CAS(y, x)$ 

```

We construct β' by iterating over the operations of p in $\min_p(\alpha')$, so that β' contains only the operations from $\min_p(\alpha')$ that appear before the operation by p . Since $\min_p(\alpha') \sim_p \alpha$ and α is the linearization of β , both $\min_p(\alpha')$ and β contain the exact same operations by process p , appearing in the exact same order.

If the next operation of p is a WRITEMAX operation, we simply append it to β' . Since each CAS returns no response, process p cannot distinguish between these executions, despite the fact that each CAS attempt might yield a different outcome and modify the memory differently.

If the next operation of p is a READMAX operation returning x , then since $\min_p(\alpha')$ respects the sequential specification, there is a preceding WRITEMAX(x) in $\min_p(\alpha')$, and no preceding WRITEMAX in $\min_p(\alpha')$ has argument $y > x$. By construction, the latter also holds in β' .

We consider the WRITEMAX operation with the maximum argument (if it exists) in β' . If the argument for this WRITEMAX is x , we simply append the READMAX operation by p to β' . Otherwise, there must be some WRITEMAX(x) operation by process $q \neq p$ that precedes the READMAX operation in $\min_p(\alpha')$. We first append a complete WRITEMAX(x) by process q , and then append the READMAX operation by p to β' .

Since x is the maximal argument of any WRITEMAX operation in β' , the algorithm guarantees that R holds the value x when p reads it during its READMAX operation.

By construction, $\beta' \sim_p \beta$. In addition, due to the minimality of $\min_p(\alpha')$, any operation not executed by p exists solely to justify a value returned by a READMAX operation by p . Hence, $\min_p(\alpha')$ does not contain any operation that is not included in β' . ◀